

Industrial Internship Report

on

Secure Vault – Python Password Manager

Prepared by

Kapil Kant Bansal

Python Internship – UpSkill Campus / The IoT Academy / UCT

GitHub: <https://github.com/kapilkantbansal/upskillcampus>

Executive Summary

This report presents Secure Vault, a command-line password manager implemented in Python. The project focuses on protecting stored credentials through layered authentication, password hashing, password-based key derivation, and authenticated encryption.

The application uses bcrypt for hashing the master password, Argon2id for deriving a 32-byte encryption key from the authenticated master password and salt, and AES-256-GCM for encrypting stored website passwords. The encryption key is derived at runtime rather than stored as a persistent secret. The credential vault is stored locally in JSON format and is intentionally excluded from version control.

The application supports adding credentials, viewing saved websites, searching for a specific website, deleting credentials, and exiting the program. Authentication, encryption, storage, and application logic are separated into individual Python modules.

1. Preface

The internship provided an opportunity to apply Python programming concepts to a practical software project. Secure Vault was developed as a Python-based password manager to address the problem of storing multiple credentials without keeping website passwords as readable plaintext.

The work involved Python modules and functions, JSON file handling, exception handling, secure password input, bcrypt, Argon2id, AES-GCM, GitHub repository management, and safe handling of sensitive files.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd

The supplied internship template describes UniConverge Technologies Pvt Ltd (UCT) as a company working in the Digital Transformation domain and providing industrial solutions with a focus on sustainability and return on investment. It identifies technologies across its ecosystem including IoT, Cyber Security, Cloud Computing, Machine Learning, communication technologies, Java Full Stack, Python and front-end technologies.

2.2 About UpSkill Campus

The supplied template states that UpSkill Campus, together with The IoT Academy and in association with UniConverge Technologies, facilitated the internship process. It describes UpSkill Campus as a career-development platform supporting learning, internships, projects, interaction with industry experts, and career-growth services.

2.3 Objective

- Gain practical experience in Python software development.
- Apply programming concepts to a complete usable application.
- Understand secure handling of authentication credentials and stored data.
- Learn how password hashing, key derivation and authenticated encryption work together.
- Practice modular design, persistence, testing, documentation and GitHub submission.
- Improve problem-solving and software-development skills.

2.4 Reference

- UpSkill Campus / UCT internship report template.
- Project repository: <https://github.com/kapilkantbansal/Secure-Valut-Python>
- Submission repository: <https://github.com/kapilkantbansal/upskillcampus>

2.5 Glossary

Term	Meaning
------	---------

AES-256-GCM	Authenticated encryption using AES with a 256-bit key.
Argon2id	Password hashing/key-derivation scheme designed to make brute-force attacks more expensive.
bcrypt	Password hashing function used for the master-password verifier.
CLI	Command Line Interface.
JSON	Structured text format used for local vault persistence.
Nonce	A value used once with encryption; this project uses a fresh 12-byte AES-GCM nonce.

3. Problem Statement

Managing multiple online accounts creates a practical security problem: users need to remember many credentials, while storing those credentials in plain text exposes them if the storage file is accessed by an unauthorized person.

Secure Vault addresses this problem by providing a local command-line password manager that authenticates the user with a master password and stores individual website passwords in encrypted form. The master password is not stored as plaintext, and the encryption key is derived during an authenticated session rather than stored as a separate persistent secret.

4. Existing and Proposed Solution

4.1 Existing Approaches and Limitations

A basic password-storage program may write usernames and passwords directly to a text or JSON file. Such a design is simple but exposes credentials to anyone who obtains the file. Another weak design is to store a reusable encryption key beside the encrypted data, reducing the protection provided by encryption.

4.2 Proposed Solution

Secure Vault uses layered protection. The master password is hashed using bcrypt and verified during login. A random salt is used for key derivation. After successful authentication, Argon2id derives a 32-byte key from the master password and salt. Individual website passwords are encrypted using AES-256-GCM with a fresh 12-byte nonce.

4.3 Value Addition

- Website passwords are not stored as readable plaintext.
- The master password is protected with a one-way password hash.
- The encryption key is derived at runtime instead of being stored separately.
- AES-GCM provides confidentiality and integrity protection for encrypted password values.
- Authentication, encryption, storage and application logic are separated into modules.
- The actual vault file is excluded from GitHub through .gitignore.

4.4 Code Submission

<https://github.com/kapilkantbansal/upskillcampus>

Original project: <https://github.com/kapilkantbansal/Secure-Valut-Python>

4.5 Report Submission

https://github.com/kapilkantbansal/upskillcampus/blob/main/Secure_Vault_Kapil_USC_UCT.pdf

5. Proposed Design / Model

The application is organized into user interaction, authentication, cryptographic processing and persistent storage. The high-level flow is: start application → initialize storage → create/verify master password → derive session key → perform credential operation → save/read encrypted vault → exit.

5.1 High Level Diagram

User / CLI	Authentication	Argon2id	AES-256-GCM	JSON Vault	Operations
Input/menu	bcrypt verify	32-byte key	Encrypt/decrypt	Local data	Add/view/search/delete

5.2 Low Level Diagram

storage.py creates, loads and saves the JSON vault. auth.py handles master-password creation and verification and obtains the session key. encryption.py generates salts, derives keys using Argon2id and performs AES-GCM encryption/decryption. main.py provides the CLI menu and invokes credential operations.

5.3 Interfaces

- CLI menu: Add Credentials, View Credentials, Search Credentials, Delete Credentials, Exit.
- Input interface: website name, username, password and master password.
- Storage interface: local vault.json file represented as JSON.
- Cryptographic interface: encrypt_password() and decrypt_password().
- Authentication interface: authenticate(), hash_master_password() and verify_master_password().

6. Performance Test

Testing focused on functional correctness and security-oriented behavior rather than synthetic throughput benchmarks. For a small local CLI application, correct authentication, encryption/decryption, persistence and credential operations are more relevant than high-volume transaction throughput.

6.1 Test Plan / Test Cases

ID	Test / Condition	Expected Result
TC-01	First-run initialization	A default vault is created and the user is prompted to create a master password.
TC-02	Correct master password	Authentication succeeds and a session key is derived.
TC-03	Incorrect master password	Authentication fails and access is not granted.
TC-04	Add credential	The credential is saved and the website password is stored in encrypted form.
TC-05	Search credential	The username is displayed and the password is decrypted for the authenticated session.
TC-06	Delete credential	The selected website entry is removed from the vault.
TC-07	Persistence	Previously saved entries remain available after restarting and re-authenticating.
TC-08	Sensitive file protection	vault.json is excluded from the Git-tracked project through .gitignore.

6.2 Test Procedure

The application should be tested using controlled dummy credentials rather than real passwords. The main scenarios are initialization, successful and failed authentication, adding and searching credentials, deletion, persistence across sessions, and verifying that the actual vault file is not included in the GitHub repository.

6.3 Performance Outcome

The implementation provides the intended core password-management workflow and demonstrates the planned security architecture. No numerical latency or throughput benchmark is claimed because such measurements were not recorded as part of the project.

7. My Learnings

The project strengthened my understanding of Python beyond basic syntax by requiring multiple modules to work together as one application.

A major learning outcome was understanding the difference between password hashing and encryption. The master password does not need to be recovered, so bcrypt is used for its verifier. Website passwords must be recovered for the user after authentication, so they are encrypted rather than hashed.

I learned practical secure-development habits such as using getpass for password input, generating random salts and nonces, avoiding plaintext credential storage, keeping the encryption key out of persistent storage, and excluding sensitive vault data from GitHub.

The project improved my understanding of GitHub workflows, documentation, modular Python development, dependency management, and technical reporting.

8. Future Work Scope

- Add automated unit and integration tests using pytest.
- Improve input validation and error handling.
- Add secure password generation.
- Support editing credentials and safe master-password changes.
- Add stronger backup/recovery and vault versioning.
- Consider a desktop or web interface while preserving the cryptographic architecture.
- Add security auditing, dependency scanning and continuous integration.

Conclusion

Secure Vault demonstrates how Python can be used to build a practical security-oriented application. The project combines modular design with bcrypt-based master-password protection, Argon2id key derivation, AES-256-GCM authenticated encryption and local JSON persistence. The current implementation is a learning project and should not be treated as a replacement for a professionally audited password manager, but it provides a strong foundation for further security and software-engineering improvements.

Appendix A – Project Structure

- auth.py – master-password creation, hashing, verification and authentication flow.
- encryption.py – salt generation, Argon2id key derivation and AES-GCM encryption/decryption.
- storage.py – creation, loading and saving of the JSON vault.
- main.py – command-line menu and credential-management operations.
- README.md – project overview and setup instructions.
- .gitignore – prevents vault.json from being committed.